🚀 PROJECT ROADMAP (JAVA STACK)

🎯 1-on-1 Mentor–Student Platform (Industry 9.5–10/10)

🎯 PROJECT GOAL (CLEAR & SAFE STATEMENT)

Build a web-based **1-on-1 mentorship platform** where a mentor and a student can:

- Join a private session
- Video call in real time
- Chat via messages
- Collaboratively edit code in a shared editor

🧠 TECH STACK (JAVA – INDUSTRY APPROVED)

🖥️ Frontend

- Next.js (React + TypeScript)
- Tailwind CSS
- Monaco Editor
- Socket.io-client / WebSocket client

⚙️ Backend (Java)

- Spring Boot (Main backend 🔥)
- Spring Web (REST APIs)
- Spring Security (Auth)
- WebSocket (STOMP)

🔄 Real-Time Layer

- Spring WebSocket + STOMP
- SockJS (fallback support)

📹 Video

- WebRTC (frontend)
- Signaling via Spring WebSocket

🗄️ Database

- PostgreSQL
- ORM:
    - Spring Data JPA (Hibernate)

🔐 Authentication

- JWT-based auth (Spring Security)
- OR Supabase Auth (optional)

🚀 Deployment

- Frontend → Vercel
- Backend → Railway / Render / Docker

🛣️ 2-WEEK ROADMAP

🔹 WEEK 1 – Backend Core + Editor

🎯 Goal:

Auth + Sessions + Real-time Editor (Java backend)

🧩 Day 1–2: Setup & Architecture

Tasks

- Setup:
  - Spring Boot project
  - Next.js frontend
- Design:
  - Architecture diagram
  - ER diagram

Spring Boot Dependencies

- Spring Web
- Spring Security
- Spring Data JPA
- PostgreSQL Driver
- WebSocket

Deliverables

✅ Backend running
✅ DB connected

🔐 Day 2–3: Authentication & Roles

Features

- Signup / Login
- Mentor / Student roles
- Protected APIs

Tasks

- Implement JWT authentication
- Configure Spring Security
- Create role-based access control

Output

✅ Auth working with roles

🧑‍💻 Day 3–5: Session Management

Features

- Create session
- Join session via link
- Start / End session

APIs

POST /api/sessions/create
POST /api/sessions/join
POST /api/sessions/end
GET /api/sessions/{id}

Entity Example

```
@Entity
class Session {
  @Id
  private String id;
  private String mentorId;
  private String studentId;
  private String status;
}
```

Output

✅ Session flow working

💻 Day 5–7: Shared Code Editor (WebSocket 🔥)

Key Tech

- Spring WebSocket (STOMP)

Endpoint

/ws
/topic/session/{id}
/app/code

Flow

- User sends code update
- Server broadcasts to session topic

Example Controller

```
@MessageMapping("/code")
@SendTo("/topic/session")
public String syncCode(String code) {
  return code;
}
```

Strategy

- Last-write-wins
- Throttle updates on frontend

Output

✅ Real-time editor working

🔹 WEEK 2 – Chat + Video + Finalization

💬 Day 8–9: Chat System

Features

- Session chat
- Message persistence

Entity

```
@Entity
class Message {
  @Id
  private String id;
  private String sessionId;
  private String senderId;
  private String message;
}
```

WebSocket Chat Flow

- `/app/chat`
- `/topic/session/{id}`

Output

✅ Chat working

📹 Day 9–11: Video Call (WebRTC)

Important Concept

👉 Java backend is ONLY for signaling

WebSocket Messages

```
{
 "type": "offer",
 "sdp": "..."
}
```

Tasks

- Exchange:
    - SDP
    - ICE candidates
- Use WebSocket for signaling

Output

✅ Video call working

🔗 Day 11–12: Integration

Tasks

- Combine:
    - Editor
    - Chat
    - Video
- Handle:
    - Disconnects
    - Reconnect logic

Output

✅ Full session working

🚀 Day 13–14: Deployment & Polish

Backend

- Build JAR:

```
mvn clean install
java -jar app.jar
```

Deploy

- Railway / Render / Docker

Frontend

- Vercel

Deliverables

- 🌐 Live URL
- 💻 GitHub repo
- 📄 README
- 🎥 Demo video

🧩 DATABASE TABLES (JAVA VERSION)

users

- id
- email
- role

sessions

- id
- mentor_id
- student_id
- status

messages

- id

- session_id
- sender_id
- message

code_snapshots (optional)

- id
- session_id
- code

🎯 WHAT MAKES THIS 9.5-10/10 (JAVA)

You can say:

> “Built a real-time mentoring platform using Spring Boot with WebSocket-based collaboration, WebRTC signaling, and secure JWT-based authentication.”

👉 This sounds **very strong for product + enterprise roles**

⚡ PRO TIPS (JAVA-SPECIFIC)

- Use layered architecture:
    - Controller → Service → Repository
- Keep DTOs separate
- Use Lombok to reduce boilerplate
- Avoid overengineering scaling

- Focus on:
    - Clean APIs
    - Stable WebSocket handling

🏆 BONUS (IF YOU WANT TO STAND OUT EVEN MORE)

- Add Redis for scaling WebSockets
- Add Docker setup
- Add API documentation (Swagger)